

Assignment 2 – LLM, AI/ML and Vector Databases

1. Self-Assessment: LLM, Deep Learning, AI and ML

I would rate my current technical proficiency as follows:

Area	Rating	Explanation
Machine Learning (ML)	A	I can independently work on ML workflows including data preprocessing, feature engineering, model training, evaluation, and prediction using Python and libraries such as Pandas and Scikit-learn.
Artificial Intelligence (AI)	A	I have hands-on understanding of AI concepts and have worked on AI-based applications involving machine learning, generative AI, and intelligent data processing.
Large Language Models (LLM)	B	I understand LLM concepts such as prompting, embeddings, RAG, context windows, hallucinations, tool calling, and LLM application architecture. I can implement LLM-based applications, while some advanced production-level LLM development would benefit from supervision.
Deep Learning	B	I understand fundamental concepts such as neural networks, training, loss functions, optimization, embeddings, and deep learning architectures. I can work with these concepts under guidance when dealing with more advanced architectures or production-scale implementations.

Summary

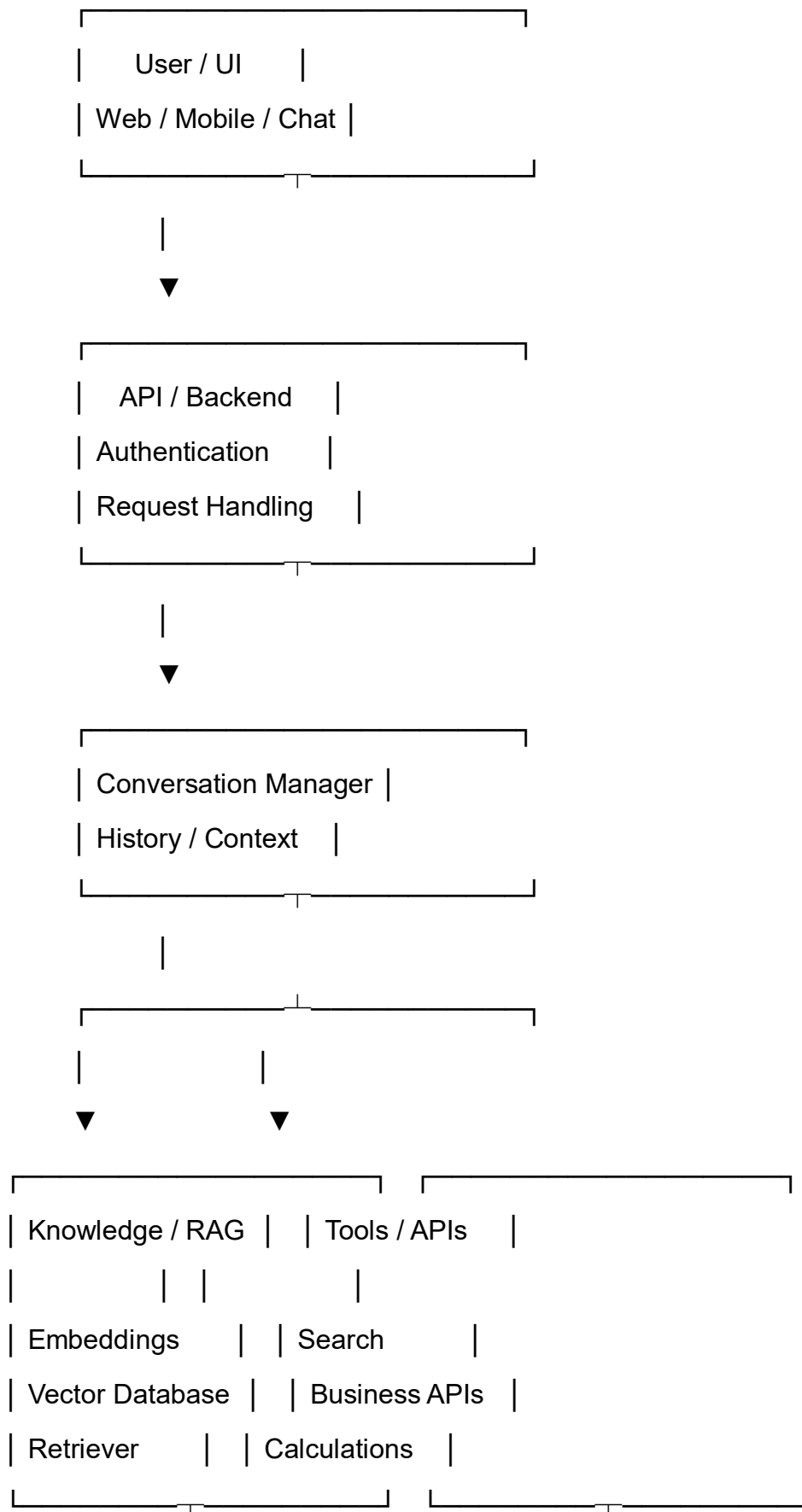
My strongest area is **Machine Learning and practical AI development**, where I am comfortable implementing complete workflows independently. I have also been developing my knowledge of **Generative AI and LLM-based applications**, particularly RAG, embeddings, prompt engineering, and AI agents.

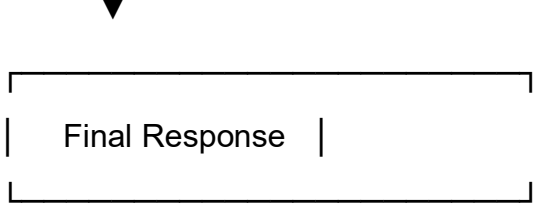
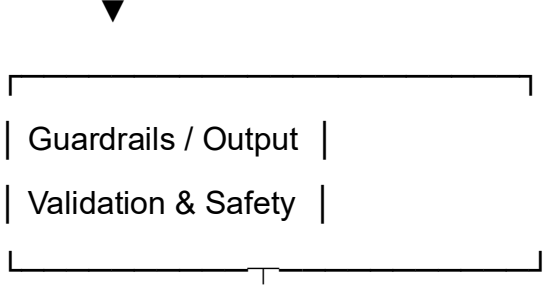
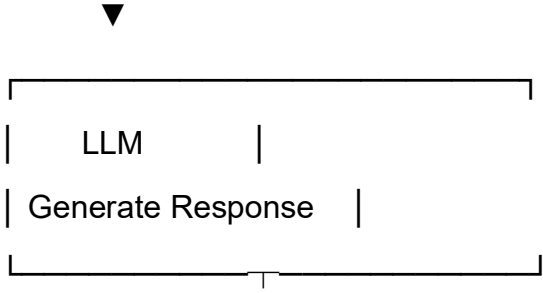
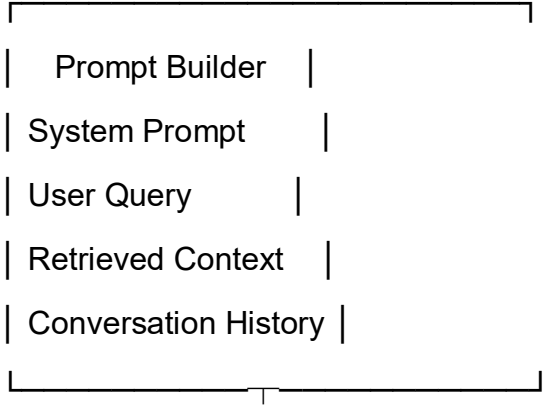
I consider the ratings a representation of my **current hands-on capability**, rather than simply theoretical knowledge. I am continuously improving my LLM and deep learning skills through practical projects and experimentation.

2. Key Architectural Components of an LLM-Based Chatbot

An LLM chatbot is more than just an LLM connected to a chat interface. A production-ready chatbot generally consists of multiple components responsible for user interaction, context management, knowledge retrieval, LLM inference, safety, and monitoring.

High-Level Architecture





2.1 User Interface

The user interacts with the chatbot through a web application, mobile application, messaging platform, or another interface.

The interface sends the user's message to the backend through an API.

Examples include:

- Web chat interface
- Mobile application
- Customer-support portal
- Internal enterprise application

2.2 Backend / API Layer

The backend acts as the central orchestration layer.

It can be responsible for:

- Authentication and authorization
- Receiving user requests
- Managing sessions
- Calling the LLM
- Calling external tools
- Connecting to databases
- Returning responses to the frontend

Python frameworks such as **FastAPI** or Flask can be used to build this layer.

2.3 Conversation and Context Management

LLMs do not automatically remember every previous interaction indefinitely.

A chatbot therefore needs a mechanism for managing conversation context.

For example:

User: What is the leave policy?

Bot: Employees receive 20 days of annual leave.

User: Can I carry it forward?

The second question depends on the previous conversation.

The application can provide relevant conversation history to the LLM as part of its context.

For longer conversations, the system can use techniques such as:

- Conversation summarization
- Sliding context windows
- Persistent conversation storage
- Retrieval of relevant previous messages

2.4 Knowledge Retrieval / RAG

A general-purpose LLM may not know an organization's private or frequently changing information.

For example, suppose we build an HR chatbot that needs to answer questions using:

- Employee handbook
- Leave policies
- Benefits documents
- Internal HR procedures

Instead of putting all documents into every prompt, we can implement **Retrieval-Augmented Generation (RAG)**.

The process is:

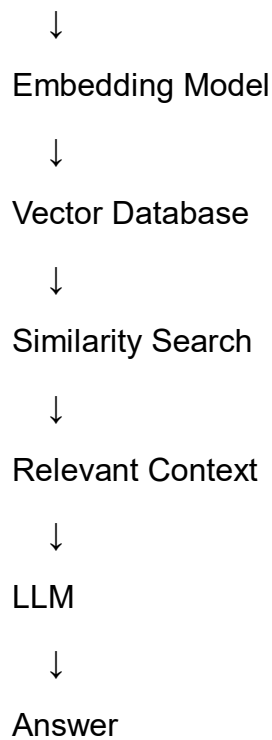
Documents



Document Processing



Chunking



When the user asks a question, the question is converted into an embedding. The system searches the vector database for semantically similar document chunks.

The retrieved information is then provided to the LLM as context.

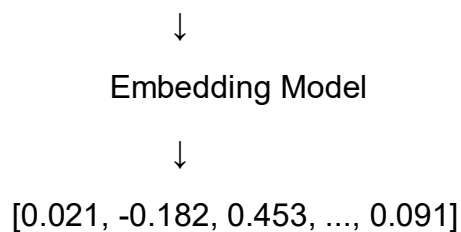
This allows the model to generate an answer grounded in the organization's data.

2.5 Embedding Model

An embedding model converts text into a numerical vector.

For example:

"How many days of annual leave do employees receive?"



Semantically similar pieces of text tend to have similar vector representations.

Embeddings are therefore useful for semantic search.

2.6 Vector Database

The generated embeddings can be stored in a vector database.

When a user asks a question:

User Query



Query Embedding



Vector Similarity Search



Top Relevant Documents

The retrieved documents are then passed to the LLM.

Vector databases are discussed in more detail in Section 3.

2.7 Prompt Builder

The prompt builder combines multiple sources of information:

System Instructions

+

Conversation History

+

User Question

+

Retrieved Knowledge

+

Tool Results



Prompt



LLM

A well-designed prompt helps the LLM understand:

- Its role

- The task
 - Available context
 - Expected output format
 - Restrictions and safety requirements
-

2.8 Tools and External APIs

Modern LLM applications can use tools rather than relying exclusively on generated text.

For example, a banking chatbot could use:

LLM

|

└─ Account Balance API

└─ Transaction API

└─ Payment API

└─ Customer Information API

The LLM determines when a tool is needed, while the backend executes the actual operation.

This is commonly referred to as **tool calling** or **function calling**.

2.9 Guardrails and Security

A production chatbot needs security and safety mechanisms.

Important considerations include:

- Authentication
- Authorization
- Input validation
- Prompt-injection protection
- Sensitive-data protection
- Output validation
- Rate limiting

- Access control
- PII protection

For example, an HR chatbot should not expose another employee's confidential information simply because a user asks for it.

2.10 Monitoring and Evaluation

A production system should be monitored continuously.

Useful metrics include:

- Response latency
- Token usage
- Cost
- Retrieval relevance
- Answer accuracy
- Hallucination rate
- User feedback
- API errors
- Tool-call failures

Evaluation can be performed using predefined test questions and human or automated evaluation.

2.11 Overall Approach

For a production LLM chatbot, I would follow this approach:

1. Define the chatbot's business objective.
2. Identify the required data sources.
3. Select an appropriate LLM.
4. Build the backend API.
5. Implement conversation/session management.
6. Add RAG if private or domain-specific knowledge is required.
7. Add embeddings and a vector database.
8. Add tools/APIs when real-time actions or information are required.

9. Implement authentication and guardrails.
 10. Test the chatbot using representative questions.
 11. Monitor quality, latency, cost, and errors.
 12. Continuously improve prompts, retrieval, and evaluation.
-

3. Vector Databases

3.1 What is a Vector?

A vector is a numerical representation of data.

In AI applications, text can be converted into vectors using an embedding model.

For example:

"Machine learning is a subset of AI"

↓

Embedding Model

↓

[0.12, -0.43, 0.67, ...]

The vector captures semantic characteristics of the original content.

3.2 What is a Vector Database?

A **vector database** is a database designed to store, index, and search high-dimensional vectors efficiently.

Traditional databases generally search structured values.

For example:

```
SELECT *
```

```
FROM products
```

```
WHERE category = 'Laptop';
```

A vector database can instead perform a semantic similarity search.

For example:

Query:

"computer for programming"



Embedding



Vector Similarity Search



Results:

"Developer laptop"

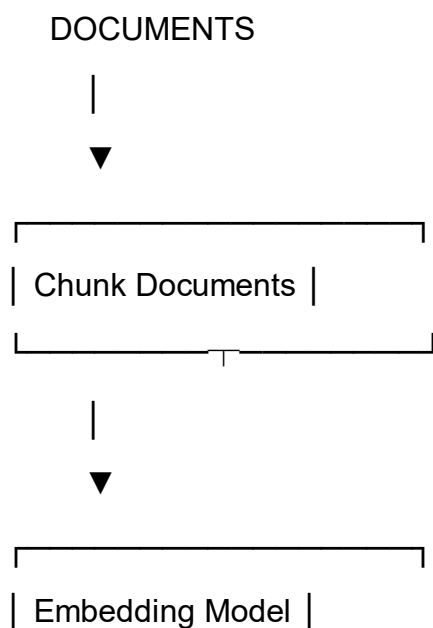
"High-performance notebook"

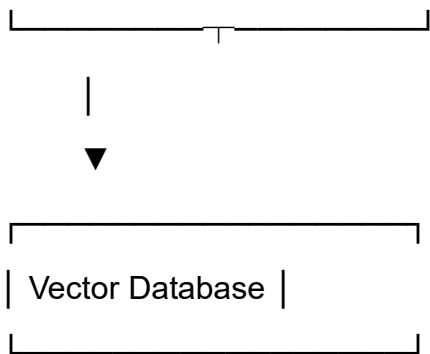
"Programming workstation"

The search is based on semantic similarity rather than exact keyword matching.

3.3 How Vector Search Works

A typical vector search pipeline looks like this:





User Query



Embedding Model



Query Vector



Similarity Search



Top-K Relevant Chunks



LLM



Final Answer

3.4 Similarity Search

Vector databases use similarity measures to identify vectors that are close to the query vector.

Common approaches include:

Cosine Similarity

Cosine similarity measures the angle between two vectors.

It is commonly used for text embeddings.

Euclidean Distance

This measures the mathematical distance between vectors.

Dot Product

This calculates the dot product between vectors and can also be used for similarity ranking.

The exact method depends on the embedding model and vector database implementation.

3.5 Example Problem

Hypothetical Problem: Enterprise HR Knowledge Assistant

Suppose a company has thousands of documents:

- HR policies
- Employee handbook
- Leave policy
- Benefits documentation
- Travel policy
- Internal procedures
- Compliance documents

Employees want to ask questions such as:

"How many days of leave can I carry forward?"

A traditional keyword search system may struggle when the question wording differs from the wording used in the documents.

A vector database can perform semantic search.

For example:

User Question

|



"What is the maximum leave
carry-forward limit?"

|



Embedding

|



Vector Search

|



Relevant HR Policy

|



LLM

|



Grounded Answer

3.6 Vector Database Options

Some commonly used vector database technologies include:

Technology General Characteristics

Pinecone Managed vector database designed specifically for vector search

Weaviate Open-source vector database with semantic search capabilities

Milvus Open-source system designed for large-scale vector search

Qdrant Vector search engine with filtering and similarity search capabilities

Technology General Characteristics

pgvector Vector similarity search extension for PostgreSQL

Chroma Lightweight option commonly used for development and experimentation

The appropriate choice depends on factors such as scale, infrastructure, existing databases, operational requirements, filtering needs, and cost.

3.7 My Choice for the Hypothetical Problem

For the HR knowledge assistant described above, I would consider **pgvector with PostgreSQL** if the organization already uses PostgreSQL as its primary database.

Reasoning

The application may need both:

Structured data

Employee ID

Department

Location

Role

Document metadata

Access permissions

and:

Vector data

Document embeddings

Using PostgreSQL with pgvector allows the application to keep structured relational data and vector representations within the same database ecosystem.

This can simplify:

- Data management
- Application architecture
- Transactions
- Access control
- Metadata filtering

- Operational maintenance

For example, vector search could be combined with metadata filtering:

Search:

"Leave carry-forward policy"

Filter:

Department = "HR"

Document Type = "Policy"

Access Level = "Employee"

This can be useful when retrieval must respect document-level permissions.

3.8 When I Would Choose a Different Vector Database

The choice is dependent on the requirements.

For example:

Requirement	Potential Choice
Existing PostgreSQL infrastructure	pgvector
Managed vector infrastructure	Pinecone
Open-source deployment	Qdrant / Weaviate / Milvus
Large-scale vector workloads	Milvus or other scalable vector systems
Lightweight experimentation	Chroma

Therefore, I would not select a vector database solely based on popularity. I would first evaluate:

1. Dataset size
2. Query volume
3. Latency requirements
4. Filtering requirements
5. Metadata requirements
6. Existing infrastructure
7. Deployment model

8. Security requirements
 9. Operational complexity
 10. Cost
-

4. Conclusion

An LLM-based chatbot is typically a combination of several components rather than just an LLM.

A practical architecture can include:

User Interface



Backend/API



Conversation Management



RAG / Tools



Prompt Construction



LLM



Guardrails



Response

For applications requiring organization-specific knowledge, **RAG and vector databases** provide an important mechanism for retrieving relevant information before generating an answer.

The choice of vector database should be based on the application's requirements rather than selecting a technology universally. For the hypothetical HR knowledge assistant, PostgreSQL with pgvector would be a reasonable architecture when PostgreSQL is already part of the organization's infrastructure because it can combine structured relational data, metadata, access controls, and vector search within the same database environment.

Overall, my approach to LLM application development is to focus not only on the model itself, but also on **data retrieval, system architecture, security, evaluation, scalability, and maintainability**.